

C++ COMPETITIVE PROGRAMMING + DATA STRUCTURES + ALGORITHMS MASTER TEXTBOOK

PART XI

Trees and BST

Traversals, height and diameter, and binary search tree operations — recursion from Part X applied to a genuinely branching structure

PART XI

Trees and BST

A hierarchical structure where recursion finally meets its natural home

Every recursive design in Part X worked on a linear or branching sequence — an array, a string, a set of choices. A tree is the first structure in this book that is inherently recursive: a tree is either empty, or it's a node with a value and two smaller trees (its left and right subtrees) hanging off it. That definition is not a teaching simplification — it is literally how every tree algorithm in this Part is written: base case at an empty (null) node, recursive case that combines whatever the left and right subtrees report back.

Contents of Part XI

- 1. What Is a Tree? Terminology and Binary Trees
- 2. Depth-First Traversals — Preorder, Inorder, Postorder
- 3. Level-Order Traversal (BFS on Trees)
- 4. Height, Diameter, and Balance
- 5. Binary Search Trees — Search and Insertion
- 6. BST Deletion, Validation, Predecessor and Successor
- 7. Common Tree Problem Patterns
- Part XI Revision, Cheat Sheet, and Interview Questions

Chapter 1 — What Is a Tree? Terminology and Binary Trees

1.1 The recursive definition

In one sentence: a binary tree is either empty, or a node holding a value with a left binary tree and a right binary tree attached to it.

That's the whole definition, and it's recursive by nature — a tree's subtree is itself a tree. This is exactly why every tree algorithm in this Part follows Part X's recursive design process directly: base case at an empty tree (a null pointer), recursive case that processes the current node and combines whatever the left and right recursive calls return.

```
struct Node {  
    int val;  
    Node* left;  
    Node* right;  
    Node(int v) : val(v), left(nullptr), right(nullptr) {}  
};
```

The standard binary tree node in C++. Every tree algorithm in this Part operates on this shape.

1.2 Terminology

Term	Meaning
Root	The single topmost node; the only node with no parent.
Parent / child	A node directly above / below another, connected by an edge.
Leaf	A node with no children (left and right are both null).
Edge	A connection between a parent and a child.
Depth of a node	Number of edges from the root down to that node.
Height of a node	Number of edges on the longest path from that node down to a leaf.

Term	Meaning
Height of the tree	Height of the root.
Subtree	Any node together with all of its descendants, viewed as a tree on its own.
Binary tree	Each node has at most two children, distinguished as left and right.
Full binary tree	Every node has either 0 or 2 children (never exactly 1).
Complete binary tree	Every level is fully filled except possibly the last, which fills left to right.
Balanced binary tree	For every node, the heights of its left and right subtrees differ by at most 1.

Depth vs. height — the classic mix-up

Depth is measured downward from the root to a node; height is measured upward from a node to its deepest leaf. The root has depth 0 and height equal to the whole tree's height; a leaf has height 0 but its depth depends on where it sits. Keep both directions straight — problems that ask for one when you compute the other are a frequent, avoidable source of wrong answers.

1.3 Building a small tree by hand

```
//          1
//         / \
//        2   3
//       / \
//      4   5
```

```
Node* root = new Node(1);
root->left = new Node(2);
root->right = new Node(3);
root->left->left = new Node(4);
root->left->right = new Node(5);
```

 Quick Revision

- A binary tree is recursively defined: empty, or a node with a left and a right subtree.
- Depth counts down from the root to a node; height counts up from a node to its deepest leaf.
- Leaf = no children. Full = every node has 0 or 2 children. Complete = filled level by level, left to right on the last level. Balanced = left/right subtree heights never differ by more than 1 at any node.
- Nearly every tree algorithm follows the same shape: base case at nullptr, recursive case combining the results from left and right.

Chapter 2 — Depth-First Traversals: Preorder, Inorder, Postorder

2.1 What a traversal is

A traversal visits every node in a tree exactly once, in some defined order. The three depth-first orders differ only in when, relative to recursing into the children, the current node itself is processed. All three share the exact same recursive shape — memorize the shape once, and the three orders are just where one line moves.

```
void preorder(Node* node) {
    if (node == nullptr) return;    // base case
    visit(node);                    // process BEFORE recursing
    preorder(node->left);
    preorder(node->right);
}

void inorder(Node* node) {
    if (node == nullptr) return;
    inorder(node->left);
    visit(node);                    // process BETWEEN the two
    // subtrees
    inorder(node->right);
}

void postorder(Node* node) {
    if (node == nullptr) return;
    postorder(node->left);
    postorder(node->right);
    visit(node);                    // process AFTER recursing
}
```

2.2 Dry run on the tree from Chapter 1

Using the tree built in Section 1.3 (root 1, children 2 and 3, and 2's children 4 and 5):

Order	Visit sequence	Rule of thumb
Preorder	1, 2, 4, 5, 3	root, then left subtree, then right subtree
Inorder	4, 2, 5, 1, 3	left subtree, then root, then right subtree
Postorder	4, 5, 2, 3, 1	left subtree, then right subtree, then root

✓ **The single most useful special case: inorder on a BST**

Running inorder traversal on a binary search tree (Chapter 5) visits every value in sorted order — always, for any valid BST. This is one of the most frequently exploited facts in tree problems: “print sorted”, “kth smallest”, and “validate BST” problems very often reduce to “run an inorder traversal and check/use the sequence it produces.”

2.3 Why postorder matters for computed properties

Postorder visits both children before the parent, which is exactly the order you need whenever a node's answer depends on its children's answers — height, diameter, subtree sum, and balance checks (Chapters 4 and 7) are all naturally postorder computations, even when the code doesn't call a function literally named postorder.

2.4 Iterative traversal with an explicit stack

Every recursive traversal can be rewritten iteratively using an explicit stack (Part IX) — this is worth knowing because recursion depth is bounded by the call stack (Part X, Chapter 1), and a very unbalanced tree (essentially a linked list) can recurse deep enough to risk a stack overflow.

```
vector<int> preorderIterative(Node* root) {
    vector<int> result;
    if (root == nullptr) return result;
    stack<Node*> st;
    st.push(root);
    while (!st.empty()) {
        Node* node = st.top(); st.pop();
        result.push_back(node->val);
        // push right BEFORE left, so left is popped (processed) first
        if (node->right) st.push(node->right);
        if (node->left) st.push(node->left);
    }
    return result;
}
```

⚠ Common mistake: swapping the push order

Because a stack is LIFO, whichever child is pushed last comes out first. To visit left before right (as preorder requires), push right first, then left. Getting this backwards silently produces a mirror-image traversal that's easy to miss without a concrete dry run.

Practice — Chapter 2

Problem 2.1 — Implement all three traversals iteratively [Level 3]

Write iterative (non-recursive) versions of preorder, inorder, and postorder using an explicit stack.

Hint 1: Preorder is the easiest to convert directly (Section 2.4).

Hint 2: Inorder needs a different technique: push left children onto the stack until null, then pop, visit, and move right.

Hint 3: Postorder is often done by computing a modified preorder (root, right, left) and reversing the result.

Key observation: Postorder's push-and-reverse trick works because postorder is the exact reverse of (root, right, left) order — recognizing that relationship avoids needing a genuinely different algorithm.

Complexity: $O(n)$ time, $O(h)$ extra space where h is the tree's height.

Problem 2.2 — Reconstruct a tree from traversals [Level 4]

Given a tree's preorder and inorder traversal sequences (with distinct values), reconstruct the original tree.

Hint 1: The first element of preorder is always the root.

Hint 2: Find that root's value in the inorder sequence — everything to its left in inorder is the left subtree; everything to its right is the right subtree.

Key observation: Knowing the size of the left subtree from the inorder split tells you exactly how to split the remaining preorder sequence too — the first (`leftSize`) elements after the root are the left subtree's preorder, and the rest are the right subtree's.

Approach: Use a hash map from value to inorder index for $O(1)$ lookups, and recurse on the computed subarray ranges rather than physically slicing arrays.

Complexity: $O(n)$ with a hash map for inorder lookups; $O(n^2)$ if searching inorder linearly each time.

Chapter 3 — Level-Order Traversal (BFS on Trees)

3.1 Why this needs a queue, not a stack

Level-order traversal visits nodes level by level, left to right within each level — the root, then both its children, then all four grandchildren, and so on. This is breadth-first search (Part IX, Chapter 6), and exactly like graph BFS, it needs a queue: nodes must be processed in the exact order they were discovered, which is a FIFO requirement, not a LIFO one.

```
vector<vector<int>> levelOrder(Node* root) {
    vector<vector<int>> result;
    if (root == nullptr) return result;
    queue<Node*> q;
    q.push(root);
    while (!q.empty()) {
        int levelSize = q.size();           // snapshot: exactly th
is
        level's nodes
        vector<int> level;
        for (int i = 0; i < levelSize; i++) {
            Node* node = q.front(); q.pop();
            level.push_back(node->val);
            if (node->left) q.push(node->left);
            if (node->right) q.push(node->right);
        }
        result.push_back(level);
    }
    return result;
}
```

3.2 Why levelSize is captured before the inner loop

This is the single detail that makes level-order traversal produce results grouped by level rather than one flat list. `q.size()` is read once, into `levelSize`, before any children of the current level are pushed — so the inner loop processes exactly the nodes that were already in the queue

(the current level), even though new nodes (the next level) are being pushed onto the same queue during that very loop.

3.3 Dry run

Same tree as Chapter 1 (root 1; children 2, 3; 2's children 4, 5):

Queue at start of outer loop	levelSize	level collected
[1]	1	[1]
[2, 3]	2	[2, 3]
[4, 5]	2	[4, 5]

Result: `[[1], [2, 3], [4, 5]]` — three levels, exactly matching the tree's shape.

✓ Recognizing when a tree problem wants level order

Clue words: “level by level,” “per-level average/sum/max,” “zigzag traversal,” “right side view,” “minimum depth” (the first leaf found by BFS gives minimum depth immediately, without exploring the whole tree — an advantage DFS doesn't have for this specific question).

Practice — Chapter 3

Problem 3.1 — Right side view of a binary tree [Level 2]

Return the value of the last (rightmost) node at each level, as if looking at the tree from the right side.

Hint 1: Run level order; for each level, the last element added is the rightmost node.

Approach: Reuse the level-order skeleton; take `level.back()` from each collected level.

Complexity: $O(n)$ time, $O(n)$ space.

Problem 3.2 — Minimum depth of a binary tree [Level 2]

Find the length of the shortest path from the root to any leaf.

Hint 1: A DFS solution must explore every path to be sure it found the true minimum. A BFS solution can stop the instant it finds the first leaf.

Key observation: BFS explores level by level, so the very first leaf encountered is guaranteed to be at the minimum depth — no need to compare against other paths.

Complexity: $O(n)$ worst case for DFS; often much better than $O(n)$ for BFS since it can return early.

Chapter 4 — Height, Diameter, and Balance

4.1 Height, computed recursively

The height of a tree is 1 + the taller of its two subtrees' heights — a direct recursive definition, with an empty tree having height -1 (or 0, depending on convention; this book uses -1 for an empty tree so a single-node tree correctly comes out to height 0).

```
int height(Node* node) {  
    if (node == nullptr) return -1;           // base case: empty  
    tree  
    return 1 + max(height(node->left), height(node->right));  
}
```

This is postorder in disguise

height() computes both children's heights before combining them into its own answer — exactly the postorder shape from Chapter 2.3, even though it's never called that in the code.

4.2 Diameter: the longest path, which may not pass through the root

The diameter of a tree is the length (in edges) of the longest path between any two nodes — and critically, that path does not have to pass through the root. The key insight: for any single node, the longest path that passes through that node (as the highest point on the path) is $\text{leftHeight} + \text{rightHeight} + 2$ (the two edges connecting the node to each subtree's deepest point). The diameter of the whole tree is the maximum of this quantity over every node.

```

int best = 0;    // best diameter (in edges) found so far

int diameterHelper(Node* node) {
    if (node == nullptr) return -1;           // height of empty
    tree
    int leftHeight = diameterHelper(node->left);
    int rightHeight = diameterHelper(node->right);
    best = max(best, leftHeight + rightHeight + 2); // path through
    h
    this node
    return 1 + max(leftHeight, rightHeight);    // this node's
    own
    height
}
// call diameterHelper(root); answer is left in `best`

```

✓ Why this is $O(n)$, not $O(n^2)$

A naive approach recomputes `height(node->left)` and `height(node->right)` separately, from scratch, for every node — $O(n)$ work per node, $O(n^2)$ total. The version above computes height and updates the diameter candidate in the same postorder pass, reusing `leftHeight/rightHeight` that were already being computed anyway — $O(n)$ total. This “compute two things in one pass instead of two passes” idea reappears constantly in tree problems.

4.3 Checking if a tree is balanced

A tree is balanced if, for every node, its left and right subtree heights differ by at most 1. The naive version checks this node-by-node, recomputing height from scratch each time — $O(n^2)$ again. The efficient version, exactly like diameter above, computes height and checks balance in a single postorder pass, using -1 (via a sentinel value) to signal “already found unbalanced, stop early.”

```

int checkBalanced(Node* node) {
    if (node == nullptr) return -1;           // height of empty
    tree
    int leftHeight = checkBalanced(node->left);
    if (leftHeight == -2) return -2;          // already
    unbalanced below
    int rightHeight = checkBalanced(node->right);
    if (rightHeight == -2) return -2;         // already
    unbalanced below
    if (abs(leftHeight - rightHeight) > 1) return -2; // unbalanced
    HERE
    return 1 + max(leftHeight, rightHeight); // normal height
}
bool isBalanced(Node* root) { return checkBalanced(root) != -2; }

```

-2 is used as a dedicated “unbalanced” sentinel, distinct from -1 (the height of an empty tree) — this lets the function short-circuit and stop doing work the moment imbalance is found anywhere below.

Practice — Chapter 4

Problem 4.1 — Sum of all node values [Level 0]

Compute the sum of every value in a binary tree.

Hint 1: Base case: empty tree sums to 0. Recursive case: node's value + sum(left) + sum(right).

Complexity: $O(n)$ time.

Problem 4.2 — Maximum path sum [Level 5]

Find the maximum sum along any path between two nodes in the tree (the path need not pass through the root, and node values may be negative).

Hint 1: Same structural idea as diameter: for each node, the best path THROUGH it is $\text{node} \rightarrow \text{val} + \max(0, \text{leftGain}) + \max(0, \text{rightGain})$, where a negative subtree contribution is simply excluded (treated as 0).

Hint 2: The value RETURNED to the parent must be a single-sided path (node->val + the better of leftGain/rightGain, floored at 0) — a path can't branch both ways once it continues upward.

Key observation: Two different quantities are in play: the best 'through this node' path (used only to update a global best answer) and the best 'extending upward from this node' path (returned to the parent) — conflating them is the most common bug in this problem.

Complexity: $O(n)$ time with the single-pass postorder technique.

Chapter 5 — Binary Search Trees: Search and Insertion

5.1 The BST invariant

In one sentence: a binary search tree is a binary tree where, for every node, every value in its left subtree is smaller and every value in its right subtree is larger. This single invariant, maintained at every node, is what makes search, insertion, and deletion all run in $O(\text{height})$ time — at each step, comparing against the current node tells you which entire subtree to ignore.

5.2 Search

```
Node* search(Node* node, int target) {
    if (node == nullptr || node->val == target) return node;
    if (target < node->val) return search(node->left, target);
    return search(node->right, target);
}
```

Every comparison eliminates one entire subtree from consideration — this is the same halving idea from Part X's power function and Part VI's binary search, applied to a tree shape instead of an array.

5.3 Insertion

```
Node* insert(Node* node, int val) {
    if (node == nullptr) return new Node(val);    // found the spot
    ;
    create it
    if (val < node->val) node->left = insert(node->left, val);
    else if (val > node->val) node->right = insert(node->right, val);
    // if val == node->val, this book's convention is: do nothing
    (no duplicates)
    return node;
}
// call as: root = insert(root, val);
```

✎ Why the return value matters

`insert()` returns the (possibly newly created) node so that `node->left = insert(node->left, val)` correctly reattaches a freshly created node to its parent. Without capturing and reassigning the return value at every level, the newly created node would be created and then immediately lost.

5.4 Dry run: inserting into a BST

Starting from an empty tree, insert 5, 3, 8, 1, 4 in that order:

Inse rt	Path taken	Resulting tree shape
5	empty → becomes root	5
3	5 (3<5, go left) → empty → new node	5 with left child 3
8	5 (8>5, go right) → empty → new node	5 with right child 8
1	5→3 (1<5, 1<3) → empty → new node	3's left child is 1
4	5→3 (4<5, 4>3) → empty → new node	3's right child is 4

Final tree: root 5, left subtree rooted at 3 (children 1 and 4), right child 8. An inorder traversal of this tree gives 1, 3, 4, 5, 8 — sorted, exactly as Chapter 2's tip promised.

⚠ Insertion order changes the tree's shape drastically

Inserting 1, 2, 3, 4, 5 in that already-sorted order produces a completely skewed tree — every node has only a right child, which is structurally a linked list. Search, insert, and delete all become $O(n)$ instead of $O(\log n)$ on such a tree. Part XVIII's self-balancing structures (or simply choosing a balanced BST variant) exist specifically to prevent this worst case.

Practice — Chapter 5

Problem 5.1 — Build a BST from a list of insertions and print it sorted [Level 1]

Insert a sequence of values into a BST, then print an inorder traversal.

Hint 1: Use the `insert()` function repeatedly, then reuse `inorder()` from Chapter 2.

Complexity: $O(n \times h)$ to build (h = height, which is $O(\log n)$ balanced or $O(n)$ skewed); $O(n)$ to print.

Problem 5.2 — Range sum of a BST [Level 2]

Given a BST and a range `[low, high]`, sum all node values that fall within the range.

Hint 1: Use the BST property to prune: if `node->val < low`, the entire left subtree is also too small — skip it entirely. Similarly for `node->val > high` on the right.

Key observation: This pruning is what separates an $O(h + k)$ BST-aware solution (k = nodes in range) from an $O(n)$ plain-tree traversal that checks every node regardless of structure.

Complexity: $O(h + k)$ where h is height and k is the number of nodes actually in range.

Chapter 6 — BST Deletion, Validation, Predecessor and Successor

6.1 Deletion: three cases

Deleting a node from a BST has to preserve the BST invariant afterward, and the correct fix depends on how many children the node being deleted has.

Case	Situation	Fix
0 children	Node is a leaf	Simply remove it (return nullptr to the parent).
1 child	Node has only a left or only a right child	Replace the node with its one child.
2 children	Node has both children	Replace the node's value with its inorder successor (or predecessor), then delete that successor from the right subtree.

```

Node* findMin(Node* node) {           // leftmost node = smallest
    value
    while (node->left != nullptr) node = node->left;
    return node;
}

Node* deleteNode(Node* node, int key) {
    if (node == nullptr) return nullptr;
    if (key < node->val) { node->left = deleteNode(node->left, k
ey);
    return node; }
    if (key > node->val) { node->right = deleteNode(node->right, k
ey);
    return node; }

    // key == node->val: this is the node to delete
    if (node->left == nullptr) return node->right; // 0 or 1 ch
ild
    (right)
    if (node->right == nullptr) return node->left; // 1 child
    (left)

    // two children: replace with inorder successor (smallest in r
ight
    subtree)
    Node* successor = findMin(node->right);
    node->val = successor->val;
    node->right = deleteNode(node->right, successor->val);
    return node;
}

```

Why the inorder successor specifically

The inorder successor (smallest value in the right subtree) is guaranteed to be larger than everything in the left subtree and smaller than everything else in the right subtree — copying its value up preserves the BST invariant automatically. The inorder predecessor (largest value in the left subtree) works by the same logic and is an equally valid choice.

6.2 Validating a BST

A common bug: checking only that `node->left->val < node->val < node->right->val` at every node. This is not sufficient — it only checks immediate children, not the full-subtree constraint. A node deep in the left subtree could still violate the invariant relative to an ancestor several levels up. The correct approach passes down a valid range that narrows at every step.

```
bool isValidBST(Node* node, long long lo, long long hi) {
    if (node == nullptr) return true;
    if (node->val <= lo || node->val >= hi) return false;
    return isValidBST(node->left, lo, node->val) &&
           isValidBST(node->right, node->val, hi);
}
// call as: isValidBST(root, LLONG_MIN, LLONG_MAX)
```

⚠ Why long long, not int, for the bounds

Using `INT_MIN/INT_MAX` as the initial bounds fails on a tree that actually contains a node equal to `INT_MIN` or `INT_MAX`, since the check is strict (`<=`, `>=`). Using long long bounds wider than any possible int value sidesteps this edge case entirely — the same overflow-awareness habit from Part I, Chapter 3.

6.3 Inorder predecessor and successor

The inorder successor of a node is the next value that would appear after it in an inorder traversal — the smallest value greater than it. Two cases, both exploiting the BST invariant directly rather than running a full traversal:

```

Node* inorderSuccessor(Node* root, Node* target) {
    Node* successor = nullptr;
    Node* cur = root;
    while (cur != nullptr) {
        if (target->val < cur->val) {
            successor = cur;          // cur is a POSSIBLE answer;
            left for a closer one
            cur = cur->left;
        } else {
            cur = cur->right;          // cur is too small, successor
                                     must be further right
        }
    }
    return successor;
}

```

O(h) — no traversal of the whole tree is needed, exactly like search in Chapter 5.

Practice — Chapter 6

Problem 6.1 — Kth smallest element in a BST [Level 2]

Find the kth smallest value in a BST.

Hint 1: Inorder traversal visits values in sorted order (Chapter 2) — the kth value visited is the answer.

Hint 2: You don't need to collect the whole traversal into a vector first; a counter that stops early is more efficient.

Approach: Run inorder traversal with a counter; return the value when the counter reaches k.

Complexity: $O(h + k)$ if traversal stops early once found; $O(n)$ worst case.

Problem 6.2 — Lowest common ancestor in a BST [Level 3]

Given two node values in a BST, find their lowest common ancestor (the deepest node that is an ancestor of both).

Hint 1: Use the BST property: if both values are smaller than the current node, the LCA must be in the left subtree. If both are larger, it must be in the right subtree. Otherwise, the current node IS the LCA.

Key observation: This is a direct application of the same left/right pruning used throughout this Part — no need to build parent pointers or paths explicitly, which a general (non-BST) tree's LCA problem would require.

Complexity: $O(h)$ time.

Chapter 7 — Common Tree Problem Patterns

7.1 Path problems

A large family of tree problems ask about root-to-leaf or node-to-node paths: does a path with a given sum exist, list all such paths, find the longest path, and so on. The shared technique: pass state down the recursion (the sum accumulated so far, or the path itself as a vector), and check the condition at leaves.

```
bool hasPathSum(Node* node, int target) {  
    if (node == nullptr) return false;  
    if (node->left == nullptr && node->right == nullptr)    // leaf  
        return node->val == target;  
    int remaining = target - node->val;  
    return hasPathSum(node->left, remaining) ||  
           hasPathSum(node->right, remaining);  
}
```

✓ Passing state down vs. combining state up

Path-sum-style problems pass a value DOWN through parameters (the remaining target). Height/diameter-style problems (Chapter 4) combine values UP through return values. Recognizing which direction a problem needs is often the deciding factor in how to structure the recursive function — many bugs come from mixing the two approaches inside one function.

7.2 Subtree problems

Checking whether one tree is a subtree of another, or whether two trees are identical, both reduce to a structural-equality check applied recursively.

```
bool isSameTree(Node* a, Node* b) {
    if (a == nullptr && b == nullptr) return true;    // both empty:
    if (a == nullptr || b == nullptr) return false;    // one empty,
    // one not
    return a->val == b->val &&
           isSameTree(a->left, b->left) &&
           isSameTree(a->right, b->right);
}
```

7.3 Lowest common ancestor in a general (non-BST) tree

Without the BST ordering property, LCA can't prune by comparison — instead, search both subtrees and reason about what each search reports back.

```
Node* lowestCommonAncestor(Node* node, Node* p, Node* q) {
    if (node == nullptr || node == p || node == q) return node;
    Node* left = lowestCommonAncestor(node->left, p, q);
    Node* right = lowestCommonAncestor(node->right, p, q);
    if (left && right) return node;    // p and q found in DIFFERENT
    // subtrees
    return left ? left : right;    // both found on the same
    // side, or neither
}
```

Why finding p or q in a subtree returns immediately

If the current node itself is p or q, it's returned right away without looking further — the search doesn't need to distinguish “found p” from “the LCA turned out to be p itself”, because in the case where p is an ancestor of q, p correctly IS the LCA.

7.4 Recognizing which pattern a problem needs

Clue in the problem	Likely pattern	Direction of data flow
"path from root to leaf", "path sum"	Path problem	Down (accumulate as you descend)
"height", "diameter", "balanced", "subtree sum"	Metric problem	Up (combine children's results)
"same tree", "subtree of", "mirror/symmetric"	Structural comparison	Two trees compared node by node
"lowest common ancestor"	LCA search	Up (report what was found below)
"level", "per row", "right/left view"	Level-order (BFS)	Across (one level at a time)

Quick Revision

- Every DFS traversal shares one recursive shape; preorder/inorder/postorder differ only in when the current node is processed relative to the two recursive calls.
- Inorder traversal of a BST always yields sorted order — the single most exploited BST fact.
- Level order needs a queue and the "snapshot `q.size()` before the inner loop" trick to group nodes by level.
- Height/diameter/balance problems should combine both children's results in a single postorder pass ($O(n)$), not recompute height separately for each node ($O(n^2)$).
- BST search/insert/delete all run in $O(\text{height})$ by using the ordering invariant to eliminate one whole subtree per step.
- Path problems typically push state down through parameters; metric and LCA problems typically pull state up through return values.

Practice — Chapter 7

Problem 7.1 — Check if two trees are mirror images [Level 2]

Given two binary trees, check if one is the mirror reflection of the other.

Hint 1: Adapt `isSameTree`: compare `a`'s left with `b`'s right, and `a`'s right with `b`'s left, instead of matching sides directly.

Complexity: $O(n)$ time.

Problem 7.2 (harder) — Serialize and deserialize a binary tree

[Level 4]

Design an algorithm to convert a binary tree to a string, and back to the original tree structure.

Hint 1: Preorder traversal, with an explicit marker (e.g. "#") for null children, contains enough information to reconstruct the tree uniquely — unlike preorder alone without null markers.

Key observation: Problem 2.2 of this Part reconstructed a tree from TWO traversals with no null markers; this problem shows that ONE traversal suffices if null children are explicitly recorded instead of omitted.

Complexity: $O(n)$ time and space for both serialize and deserialize.

Part XI — Master Revision

Must Remember

Quick Revision

- A binary tree is recursively defined; nearly every algorithm here follows base case = nullptr, recursive case = combine left and right.
- Preorder: root, left, right. Inorder: left, root, right. Postorder: left, right, root. Inorder on a BST always yields sorted order.
- Level order needs a queue, not a stack, and the q.size()-snapshot trick to separate levels.
- Compute height/diameter/balance in a single postorder pass to stay $O(n)$; recomputing height per node is a common $O(n^2)$ trap.
- BST operations are $O(\text{height})$: $O(\log n)$ on a balanced tree, but $O(n)$ on a skewed one — insertion order determines shape.
- BST deletion of a two-child node: replace with the inorder successor (or predecessor), then delete that successor from its original position.
- Validating a BST requires a full valid RANGE passed down, not just a comparison with immediate children.

Common Mistakes Recap

Mistake	Fix
Confusing depth (from root) with height (to deepest leaf)	Depth counts down from root; height counts up from a node
Recomputing height() separately for every node when checking balance/diameter	Combine the height computation and the check into one postorder pass
Validating a BST by comparing only immediate children	Pass a valid (lo, hi) range down through the recursion instead
Using INT_MIN/INT_MAX as BST-validation bounds	Use long long bounds wider than any possible node value
Losing a newly inserted BST node because the return value wasn't reassigned	Always write <code>node->left = insert(node->left, val)</code> , not just <code>insert(node->left, val)</code>

Mistake	Fix
Pushing left before right in an iterative preorder stack traversal	Push right first, then left, since a stack reverses the pop order
Recurring on a very unbalanced (skewed) tree with large n	Be aware of the stack-overflow risk (Part X, Chapter 1); consider an iterative rewrite if needed

Complexity Table

Operation	Balanced tree	Skewed (worst-case) tree
DFS traversal (pre/in/postorder)	$O(n)$	$O(n)$
Level-order (BFS) traversal	$O(n)$	$O(n)$
Height / diameter / balance check (single pass)	$O(n)$	$O(n)$
BST search / insert / delete	$O(\log n)$	$O(n)$
Recursion stack space during any traversal	$O(\log n)$	$O(n)$

Interview-Style Questions

- Why does inorder traversal of a binary search tree always produce a sorted sequence?
- Explain why computing height and checking balance in the same postorder pass is asymptotically better than checking balance at every node with a separate `height()` call.
- Why is a valid-range check (not just a comparison with immediate children) necessary to correctly validate a BST?
- Why does BST deletion of a two-child node use the inorder successor (or predecessor) instead of, say, an arbitrary node from either subtree?

- Why does level-order traversal require a queue, while depth-first traversals can use either recursion (the implicit call stack) or an explicit stack?

Cheat Sheet — Part XI

```
struct Node { int val; Node *left, *right; Node(int
v):val(v),left(0),right(0){} };

// DFS traversals
void preorder(Node* n){ if(!n) return; visit(n); preorder(n->left)
;
preorder(n->right); }
void inorder(Node* n) { if(!n) return; inorder(n->left); visit(n);
inorder(n->right); }
void postorder(Node* n){ if(!n) return; postorder(n->left);
postorder(n->right); visit(n); }

// Level order (BFS)
queue<Node*> q; q.push(root);
while (!q.empty()) {
    int sz = q.size();
    for (int i = 0; i < sz; i++) {
        Node* n = q.front(); q.pop();
        if (n->left) q.push(n->left);
        if (n->right) q.push(n->right);
    }
}
```

```

// Height
int height(Node* n){ if(!n) return -1; return
1+max(height(n->left),height(n->right)); }

// BST search / insert
Node* search(Node* n, int x){
    if(!n||n->val==x) return n;
    return x<n->val ? search(n->left,x) : search(n->right,x);
}
Node* insert(Node* n, int x){
    if(!n) return new Node(x);
    if(x<n->val) n->left=insert(n->left,x);
    else if(x>n->val) n->right=insert(n->right,x);
    return n;
}

// BST validate (pass a valid range down)
bool isValidBST(Node* n, long long lo, long long hi){
    if(!n) return true;
    if(n->val<=lo || n->val>=hi) return false;
    return isValidBST(n->left,lo,n->val) &&
        isValidBST(n->right,n->val,hi);
}

```

Coming up in Part XII

Part XII — Heaps takes the “parent compared to children” idea from BSTs and relaxes it into a weaker but still useful invariant: a parent only needs to be $\leq$ (or $\geq$) its children, not perfectly ordered relative to the whole tree. That relaxation is what lets a heap be stored compactly in a plain array with no pointers at all, and is exactly what powers `std::priority_queue`.